

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)
Факультет компьютерных технологий и прикладной математики
Кафедра вычислительных технологий

ОТЧЁТ
по дисциплине
«Генетические алгоритмы и иммунные системы»

**Тема: Искусственная иммунная сеть для решения задачи поиска
кратчайшего пути в графе**

Вариант 3

Работу выполнил: Сидоренко Максим
Направление подготовки: 02.04.02 Фундаментальная информатика и
информационные технологии
Преподаватель: Полупанова Е.Е.

Краснодар, 2026

Содержание

1. Введение
 2. Постановка задачи
 3. Теоретические сведения об искусственных иммунных сетях
 4. Адаптация иммунной сети к задаче кратчайшего пути
 5. Описание программной реализации
 6. Экспериментальное исследование
 7. Анализ результатов
 8. Заключение
 9. Список использованных источников
- Приложение А. Фрагменты программной реализации

1. Введение

Задача поиска кратчайшего пути в графе относится к числу классических задач дискретной оптимизации. Она используется в маршрутизации, транспортной логистике, сетевом планировании, анализе коммуникационных сетей, компьютерных играх и системах принятия решений. При небольших размерах графа задача эффективно решается точными алгоритмами, например алгоритмом Дейкстры. Однако в рамках дисциплины «Генетические алгоритмы и иммунные системы» интерес представляет не только точное решение, но и применение биоинспирированных методов оптимизации.

В данной работе реализован алгоритм искусственной иммунной сети для задачи поиска кратчайшего пути во взвешенном графе. Такой подход моделирует отдельные принципы функционирования иммунной системы: существование популяции антител, оценку аффинности, клонирование лучших антител, гипермутацию, формирование иммунной памяти и супрессию похожих решений.

Работа выполняется в паре по общему заданию: один участник реализует генетический алгоритм, второй участник реализует алгоритм иммунной сети. В данном отчёте рассматривается индивидуальная часть Сидоренко Максима: реализация искусственной иммунной сети для варианта 3 — поиска кратчайшего пути в графе.

1.1. Цель работы

Цель работы — разработать программную реализацию искусственной иммунной сети для решения задачи поиска кратчайшего пути в графе, провести экспериментальное исследование алгоритма и оценить качество найденных решений относительно эталонного алгоритма Дейкстры.

1.2. Задачи работы

1. Сформулировать математическую постановку задачи поиска кратчайшего пути в графе.
2. Описать основные понятия искусственных иммунных сетей и адаптировать их к решаемой задаче.
3. Разработать модель антитела как возможного пути между начальной и конечной вершинами.
4. Реализовать операции оценки аффинности, клонирования, гипермутации, супрессии и иммунной памяти.
5. Реализовать эталонное решение алгоритмом Дейкстры для проверки качества иммунного алгоритма.
6. Разработать средства генерации тестовых графов, визуализации результата и сохранения экспериментальных данных.
7. Провести эксперименты по времени выполнения и качеству решений.
8. Построить графики зависимости времени от размерности задачи и значения целевой функции от числа итераций.

2. Постановка задачи

Пусть задан взвешенный неориентированный граф $G = (V, E)$, где V — множество вершин, E — множество рёбер. Каждому ребру (u, v) из E поставлен в соответствие положительный вес $w(u, v)$, характеризующий стоимость перехода между вершинами u и v . Также заданы начальная вершина s и конечная вершина t .

Требуется найти путь $P = (v_0, v_1, \dots, v_k)$, такой что $v_0 = s$, $v_k = t$, а любые две соседние вершины v_i и v_{i+1} соединены ребром графа. Целевая функция равна суммарной длине пути: $F(P) = \sum w(v_i, v_{i+1})$. Необходимо минимизировать значение $F(P)$.

2.1. Входные данные

- количество вершин графа;

- список рёбер;
- вес каждого ребра;
- начальная вершина;
- конечная вершина.

2.2. Выходные данные

- найденный путь от начальной вершины к конечной;
- длина найденного пути;
- время выполнения алгоритма;
- история изменения лучшего значения целевой функции по итерациям;
- визуализация графа и найденного пути;
- таблица экспериментальных результатов.

2.3. Критерии оценки

Качество работы иммунной сети оценивается по двум основным критериям: времени выполнения и качеству получаемого решения. Для контроля качества дополнительно вычисляется эталонное решение алгоритмом Дейкстры. Отклонение найденного иммунной сетью пути от эталона рассчитывается в процентах.

Критерий	Обозначение	Назначение
Длина пути	$F(P)$	Основная целевая функция, которую необходимо минимизировать.
Время выполнения	T	Позволяет оценить вычислительную трудоёмкость алгоритма.
Отклонение от эталона	$\Delta, \%$	Показывает отличие результата ИС от результата Дейкстры.
История лучшего решения	$F_{best}(i)$	Используется для построения графика сходимости по итерациям.

Таблица 1 — Критерии оценки работы алгоритма

3. Теоретические сведения об искусственных иммунных сетях

Искусственные иммунные системы представляют собой класс биоинспирированных методов, основанных на отдельных принципах работы естественной иммунной системы. В задачах оптимизации эти методы

используют аналогию между поиском хорошего решения и реакцией иммунной системы на антиген.

Естественная иммунная система распознаёт патогены и вырабатывает антитела, способные с ними взаимодействовать. Более подходящие антитела имеют большую аффинность, активнее размножаются и сохраняются в иммунной памяти. В вычислительных алгоритмах эти идеи преобразуются в процедуры генерации решений, оценки качества, отбора, клонирования и мутации.

3.1. Основные понятия

Понятие	Биологический смысл	Смысл в данной работе
Антиген	Объект, на который реагирует иммунная система.	Задача поиска пути в конкретном графе от s до t .
Антитело	Клетка, способная распознавать антиген.	Возможный путь между начальной и конечной вершинами.
Аффинность	Степень соответствия антитела антигену.	Оценка качества пути: чем путь короче, тем аффинность выше.
Клон	Копия антитела.	Копия найденного пути, которая затем мутирует.
Гипермутация	Усиленное изменение клонов.	Перестройка участка пути для поиска более короткого варианта.
Иммунная память	Сохранение лучших антител.	Хранение лучших найденных путей.
Супрессия	Подавление избыточных похожих клеток.	Удаление одинаковых или слишком похожих путей.

Таблица 2 — Интерпретация понятий искусственной иммунной сети

3.2. Общая схема иммунной оптимизации

В общем виде иммунная оптимизация работает с популяцией клеток, каждая из которых кодирует возможное решение задачи. Для каждой клетки вычисляется качество. Лучшие клетки клонируются, после чего клоны изменяются с помощью мутаций. Если изменённый клон оказывается лучше исходного решения, он может заменить его в популяции. Для предотвращения вырождения популяции используется супрессия и добавление случайных новых решений.

9. Создать начальную популяцию антител.
10. Вычислить аффинность каждого антитела.
11. Выбрать лучшие антитела.
12. Создать клоны выбранных антител.

13. Выполнить гипермутацию клонов.
14. Сравнить клоны с исходными решениями.
15. Обновить иммунную память.
16. Выполнить супрессию похожих решений.
17. Добавить новые случайные антитела.
18. Повторять процесс до достижения критерия остановки.

4. Адаптация иммунной сети к задаче кратчайшего пути

4.1. Кодирование антитела

В данной работе антитело кодируется списком номеров вершин. Например, антитело [0, 5, 13] означает путь от вершины 0 к вершине 13 через промежуточную вершину 5. Такое представление удобно для визуализации, вычисления длины пути и выполнения мутаций.

Каждое антитело должно удовлетворять двум условиям: первая вершина пути совпадает с начальной вершиной s , последняя вершина совпадает с конечной вершиной t , а между каждой парой соседних вершин в списке существует ребро графа.

4.2. Целевая функция и аффинность

Целевая функция равна сумме весов рёбер, входящих в путь. Так как решается задача минимизации, лучшим считается антитело с меньшим значением целевой функции. Для вычислительной интерпретации аффинности используется обратная зависимость: $\text{affinity}(P) = 1 / (1 + F(P))$. Следовательно, чем меньше длина пути, тем выше аффинность.

4.3. Начальная популяция

Начальная популяция формируется случайными допустимыми путями. Для этого используется случайное блуждание по графу от начальной вершины к конечной с последующим удалением циклов. Если случайное блуждание не достигает цели за ограниченное число шагов, хвост пути достраивается с

помощью кратчайшего пути от текущей вершины до цели. Это позволяет получать допустимые антитела уже на этапе инициализации.

4.4. Клонирование и гипермутация

После вычисления качества популяция сортируется по значению целевой функции. Лучшие антитела выбираются для клонирования. Чем выше место антитела в рейтинге, тем больше клонов оно создаёт. Затем клоны подвергаются гипермутации. В реализации используются три типа мутации: перестройка хвоста пути, вставка промежуточного обхода и замена участка пути кратчайшим мостом между двумя выбранными вершинами.

4.5. Ремонт пути

После мутации путь может оказаться некорректным: например, между двумя соседними вершинами может отсутствовать ребро. Для решения этой проблемы реализован оператор ремонта. Если участок пути некорректен, он заменяется кратчайшим соединением между соответствующими вершинами. После ремонта из пути удаляются циклы.

4.6. Супрессия и иммунная память

Иммунная память хранит несколько лучших уникальных решений. Благодаря этому лучшее найденное решение не теряется при последующих итерациях. Супрессия удаляет слишком похожие антитела. Похожесть оценивается по множеству рёбер, входящих в путь. Если два пути имеют слишком большую долю общих рёбер, худший из них исключается. Это поддерживает разнообразие популяции и снижает риск преждевременной сходимости.

4.7. Псевдокод алгоритма

Вход: граф G , начальная вершина s , конечная вершина t
Выход: лучший найденный путь P_{best}

1. Сформировать начальную популяцию антител.
2. Для каждой итерации:
 - 2.1. Вычислить длину пути и аффинность каждого антитела.
 - 2.2. Отсортировать популяцию по значению целевой функции.

- 2.3. Обновить иммунную память лучшими решениями.
- 2.4. Выбрать лучшие антитела для клонирования.
- 2.5. Создать клоны выбранных антител.
- 2.6. Выполнить гипермутацию клонов.
- 2.7. Починить некорректные пути.
- 2.8. Объединить исходную популяцию, клоны и память.
- 2.9. Выполнить супрессию похожих решений.
- 2.10. Добавить случайные антитела для разнообразия.
3. Вернуть лучшее решение из иммунной памяти.

5. Описание программной реализации

Программная реализация выполнена на языке Python. Для работы с графами и визуализации используются стандартные структуры данных Python, библиотека matplotlib и библиотека networkx. Проект организован модульно, что упрощает проверку отдельных компонентов и дальнейшее объединение с генетическим алгоритмом напарника.

5.1. Структура проекта

```

Генетические алгоритмы/Мои решения/
├── README.md
├── requirements.txt
├── main.py
├── src/
│   ├── graph_model.py
│   ├── dijkstra_baseline.py
│   ├── immune_network.py
│   ├── visualization.py
│   ├── experiments.py
│   └── utils.py
├── data/
├── results/
└── report/

```

5.2. Назначение основных файлов

Файл	Назначение
main.py	Демонстрационный запуск алгоритма, вывод результата, сохранение визуализации и JSON-сводки.
src/graph_model.py	Модель взвешенного графа, загрузка и сохранение графов, генерация связанных графов, проверка путей.
src/dijkstra_baseline.py	Эталонное решение задачи кратчайшего пути методом Дейкстры.
src/immune_network.py	Основная реализация искусственной иммунной сети.
src/visualization.py	Построение изображения графа с найденным путём и графиков экспериментов.
src/experiments.py	Серия экспериментов на графах разной размерности, сохранение CSV и графиков.
results/	Папка с графиками, таблицами, скриншотами и результатами запусков.

Таблица 3 — Назначение модулей проекта

5.3. Формат входных данных

Граф хранится в формате JSON. Такой формат удобен для ручного редактирования и автоматической генерации тестовых данных. В файле указывается количество вершин, начальная вершина, конечная вершина и список рёбер с весами.

```
{
  "vertices": 8,
  "start": 0,
  "target": 7,
  "edges": [
    [0, 1, 4],
    [0, 2, 3],
    [1, 3, 2],
    [2, 3, 5],
    [3, 7, 4]
  ]
}
```

5.4. Инструкция по запуску

Для запуска проекта необходимо установить зависимости и выполнить демонстрационный запуск. Также отдельно предусмотрен запуск экспериментов.

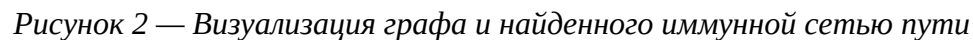
```
pip install -r requirements.txt
python main.py
python -m src.experiments
```

5.5. Демонстрационный запуск

На рисунке ниже приведён результат демонстрационного запуска программы. Искусственная иммунная сеть нашла путь, совпадающий с эталонным решением Дейкстры. Это подтверждает корректность работы алгоритма на тестовом графе.

```
=== Искусственная иммунная сеть: кратчайший путь ===
Граф: 14 вершин, 30 рёбер
Стартовая вершина: 0
Конечная вершина: 13
Путь ИС: 0 -> 5 -> 13
Длина пути ИС: 12.0000
Эталон Дейкстры: 0 -> 5 -> 13
Длина по Дейкстре: 12.0000
Отклонение: 0.00%
Время выполнения ИС: 2.689553 сек
```

Визуализация графа позволяет наглядно проверить найденный путь: на изображении отображаются вершины, веса рёбер, стартовая и конечная вершины, а также путь, найденный искусственной иммунной сетью.



Экспериментальное исследование проводилось на случайно сгенерированных связных взвешенных графах разной размерности. Для каждого графа искусственная иммунная сеть запускалась несколько раз, после чего вычислялось среднее время выполнения и средняя длина найденного пути. Для контроля качества на тех же графах запускался алгоритм Дейкстры.

Параметр	Значение/диапазон	Назначение
population_size	45–70	Количество антител в популяции.
iterations	45–140	Количество итераций оптимизации.
selection_size	часть популяции	Число лучших антител, выбранных для клонирования.
clone_multiplier	4–5	Базовое число клонов для выбранных

		антител.
mutation_rate	0.50	Вероятность гипермутации клона.
random_injection_rate	0.18	Доля случайных антител для поддержания разнообразия.
memory_size	8–10	Количество решений, сохраняемых в иммунной памяти.

Таблица 4 — Основные параметры искусственной иммунной сети

6.2. Таблица результатов

Вершин	Рёбер	Путь ИС	Длина ИС	Дейкстра	Отклонение, %	Время, сек
10	18	0 -> 6 -> 8 -> 9	20.0	20.0	0.0	0.141507
18	38	0 -> 1 -> 6 -> 17	17.0	17.0	0.0	0.242556
26	65	0 -> 13 -> 7 -> 25	28.0	28.0	0.0	0.377556
34	95	0 -> 25 -> 28 -> 5 -> 14 -> 33	28.0	28.0	0.0	0.553924

Таблица 5 — Результаты экспериментального исследования

Во всех проведённых тестах искусственная иммунная сеть нашла путь, совпадающий по длине с эталонным решением алгоритма Дейкстры. Отклонение составило 0 %, что говорит о высоком качестве найденных решений на выбранном наборе тестовых графов.

6.3. График зависимости времени от размерности задачи

На рисунке 3 показана зависимость среднего времени выполнения алгоритма от количества вершин графа. С увеличением размерности графа возрастает пространство поиска, поэтому среднее время выполнения также увеличивается.

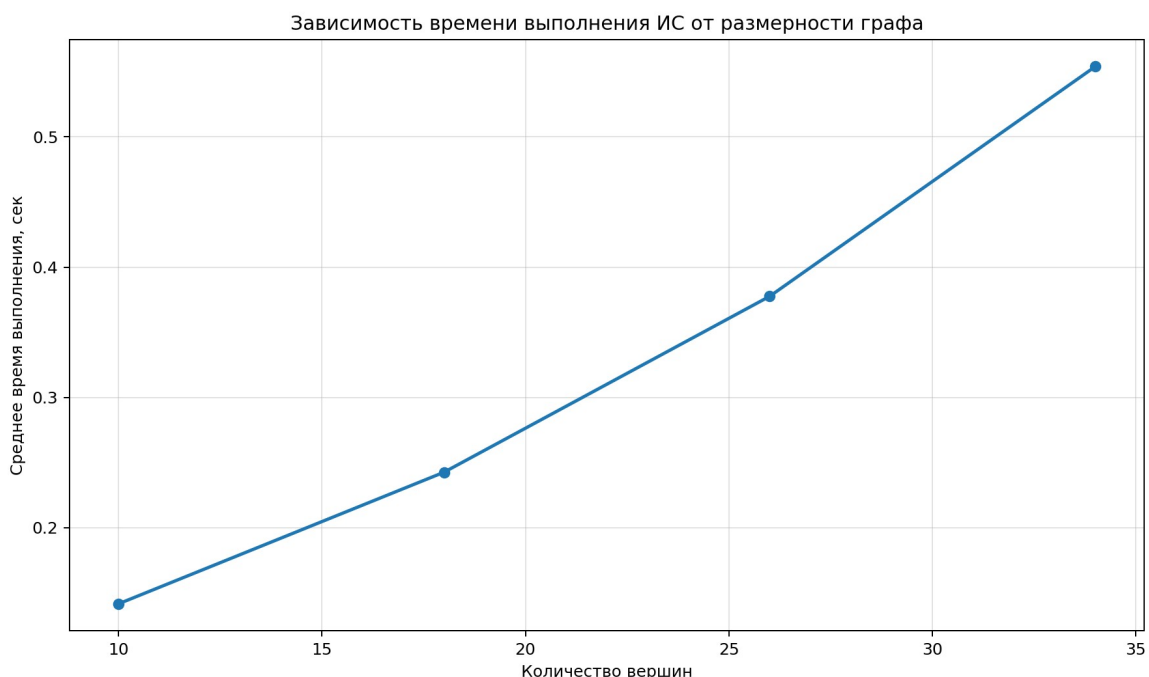


Рисунок 3 — Зависимость времени выполнения ИС от размерности графа

6.4. График значения целевой функции от числа итераций

На рисунке 4 показана динамика лучшего значения целевой функции по итерациям. Целевая функция соответствует длине найденного пути. Горизонтальная пунктирная линия соответствует эталонному значению, найденному алгоритмом Дейкстры.

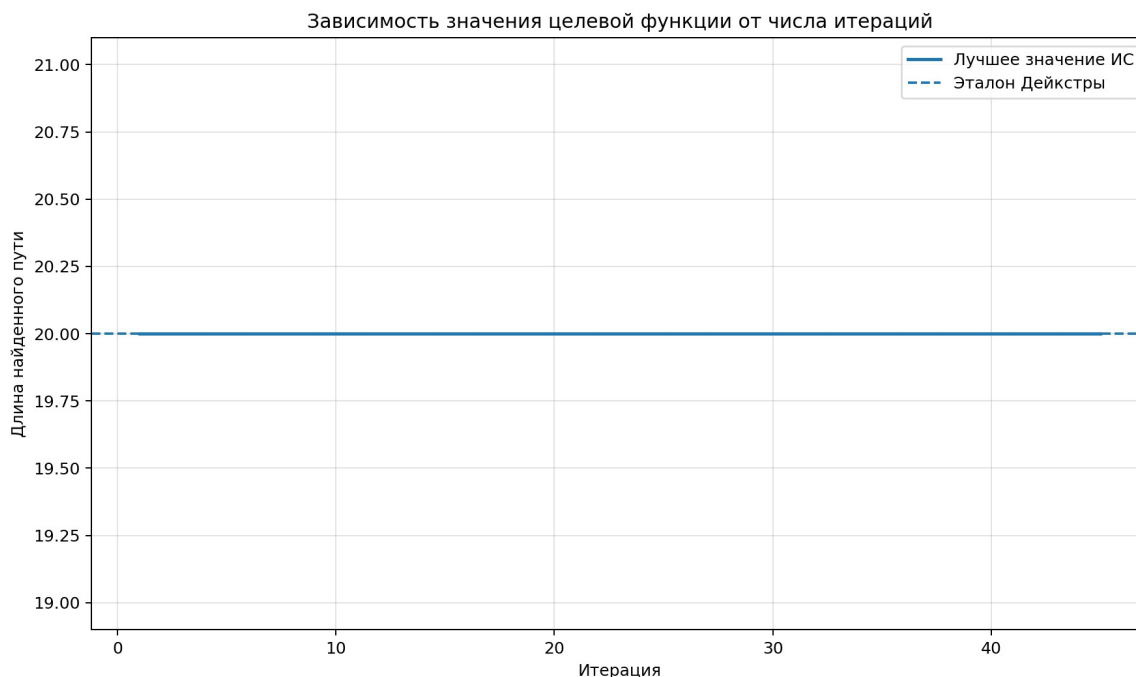


Рисунок 4 — Зависимость значения целевой функции от числа итераций

График показывает, что иммунная сеть быстро находит решение высокого качества и затем сохраняет его в памяти. Это соответствует идее иммунной памяти: лучшее найденное антитело не теряется при дальнейших мутациях и обновлениях популяции.

7. Анализ результатов

Полученные результаты показывают, что реализованная искусственная иммунная сеть корректно решает задачу поиска кратчайшего пути во взвешенном графе. На демонстрационном графе путь, найденный ИС, совпал с эталонным путём Дейкстры: длина пути составила 12.0, отклонение — 0 %.

В серии экспериментов на графах разной размерности алгоритм также находил решения, совпадающие с эталонными по длине. При этом время выполнения возрастало при увеличении числа вершин и рёбер, что является ожидаемым результатом для популяционного алгоритма.

Следует отметить, что алгоритм Дейкстры используется только как эталон качества. Основной целью работы является реализация биоинспирированного иммунного подхода, поэтому Дейкстра не заменяет иммунную сеть, а служит инструментом проверки корректности и оценки отклонения.

7.1. Достоинства реализованного подхода

- алгоритм работает с популяцией решений, а не с одним путём;
- иммунная память сохраняет лучшие найденные решения;
- гипермутация позволяет перестраивать путь и искать улучшения;
- супрессия поддерживает разнообразие решений;
- результаты можно сравнивать с генетическим алгоритмом напарника в одном формате CSV;
- реализована визуализация графа и найденного пути.

7.2. Ограничения

- алгоритм является эвристическим, поэтому на очень больших графах качество зависит от параметров популяции, числа итераций и мутации;
- время выполнения выше, чем у специализированного точного алгоритма Дейкстры;
- для получения устойчивых результатов желательно выполнять несколько запусков и усреднять показатели;
- при объединении с работой напарника необходимо использовать одинаковые тестовые графы и одинаковые стартовые/конечные вершины.

8. Заключение

В ходе работы была разработана программная реализация искусственной иммунной сети для решения задачи поиска кратчайшего пути в графе. В качестве антител использовались возможные пути от начальной вершины к конечной. Качество антител определялось длиной пути, а аффинность была обратно связана со значением целевой функции.

В алгоритме реализованы основные механизмы иммунной оптимизации: генерация начальной популяции, оценка аффинности, клонирование лучших антител, гипермутация, ремонт некорректных путей, супрессия похожих решений и иммунная память. Дополнительно реализованы загрузка и генерация графов, эталонное решение Дейкстры, визуализация найденного пути и построение экспериментальных графиков.

Экспериментальная проверка показала, что на выбранных тестовых графах искусственная иммунная сеть находит решения, совпадающие с эталонным результатом алгоритма Дейкстры. Таким образом, цель работы достигнута: реализован и исследован алгоритм искусственной иммунной сети для варианта 3 — поиска кратчайшего пути в графе.

9. Список использованных источников

1. Методические материалы по дисциплине «Генетические алгоритмы и иммунные системы». Задание ГА и ИС, 2026.
2. Конспект «ИИС.txt»: основные понятия искусственных иммунных систем, антител, антигенов, клонов и аффинности.
3. Конспект «эволюция медленное кол.txt»: модели эволюции Дарвина, Ламарка, Де Фриза, Поппера и Кимуры.
4. Конспект «Основные метода ГА 1.txt»: операторы селекции, кроссовера, мутации, инверсии и редукции.

5. Документация Python 3: стандартные структуры данных, модуль random, модуль time.
6. Документация matplotlib: построение графиков и сохранение изображений.
7. Документация NetworkX: представление и визуализация графов.

Приложение А. Фрагменты программной реализации

В приложении приведены основные фрагменты кода, отражающие реализацию искусственной иммунной сети и демонстрационного запуска программы. Полный код содержится в папке проекта.

А.1. Фрагмент класса искусственной иммунной сети

```
class ImmuneNetworkShortestPath:
    """Искусственная иммунная сеть с клональным отбором и супрессией.

    Адаптация к задаче кратчайшего пути:
    - антиген: граф и пара вершин start-target;
    - антитело: допустимый путь start -> target;
    - целевая функция: суммарная длина пути;
    - аффинность: 1 / (1 + длина пути);
    - гипермутация: перестройка участка пути;
    - супрессия: удаление одинаковых и слишком похожих путей;
    - память: лучшие найденные решения.
    """

    def __init__(
        self,
        graph: WeightedGraph,
        start: int,
        target: int,
        population_size: int = 60,
        iterations: int = 120,
        selection_size: int = 15,
        clone_multiplier: int = 4,
        mutation_rate: float = 0.45,
        suppression_threshold: float = 0.85,
        random_injection_rate: float = 0.20,
        memory_size: int = 8,
        seed: int | None = 42,
    ) -> None:
        if population_size < 4:
            raise ValueError("population_size должен быть не меньше 4.")
        if selection_size < 1 or selection_size > population_size:
            raise ValueError("selection_size должен быть в диапазоне [1,
population_size].")
        self.graph = graph
        self.start = start
        self.target = target
        self.population_size = population_size
        self.iterations = iterations
        self.selection_size = selection_size
        self.clone_multiplier = clone_multiplier
        self.mutation_rate = mutation_rate
        self.suppression_threshold = suppression_threshold
        self.random_injection_rate = random_injection_rate
        self.memory_size = memory_size
```

```

self.rng = random.Random(seed)

def run(self) -> ImmuneNetworkResult:
    """Запустить алгоритм."""
    start_time = time.perf_counter()
    population = self._initial_population()
    memory: List[Antibody] = []
    history: List[float] = []
    for _ in range(self.iterations):
        population = self._evaluate_population(population)
        population.sort(key=lambda antibody: antibody.objective)
        memory = self._update_memory(memory, population)
        history.append(memory[0].objective)
        selected = population[: self.selection_size]
        clones = self._clone_and_mutate(selected)
        clones = self._evaluate_population(clones)
        combined = population + clones + memory
        combined.sort(key=lambda antibody: antibody.objective)
        suppressed = self._suppress(combined)
        population = suppressed[: self.population_size]
        population = self._inject_random_antibodies(population)
        population = self._evaluate_population(population)
        population.sort(key=lambda antibody: antibody.objective)
        population = population[: self.population_size]
        population = self._evaluate_population(population)
        population.sort(key=lambda antibody: antibody.objective)
        memory = self._update_memory(memory, population)
        best = memory[0] if memory else population[0]
        elapsed = time.perf_counter() - start_time
        return ImmuneNetworkResult(
            best_path=best.path,
            best_length=best.objective,
            elapsed_time=elapsed,
            history=history,
            population_size=self.population_size,
            iterations=self.iterations,
            memory=memory,
        )

```

A.2. Фрагмент демонстрационного запуска

"""Демонстрационный запуск искусственной иммунной сети.

Вариант 3: поиск кратчайшего пути в графе.

Исполнитель: Сидоренко Максим.

"""

```
from __future__ import annotations
```

```
from pathlib import Path
```

```

from src.dijkstra_baseline import solve_with_dijkstra
from src.graph_model import load_graph_from_json, save_graph_to_json,
generate_connected_graph
from src.immune_network import ImmuneNetworkShortestPath
from src.utils import save_json, percent_deviation
from src.visualization import draw_graph_with_path, plot_objective_history

```

```

def ensure_demo_graph() -> Path:
    """Создать демонстрационный граф, если его ещё нет."""
    root = Path(__file__).resolve().parent
    data_dir = root / 'data'
    data_dir.mkdir(parents=True, exist_ok=True)
    graph_path = data_dir / 'graph_demo.json'
    if graph_path.exists():
        return graph_path
    graph = generate_connected_graph(vertices=14, edges=30, min_weight=1, max_weight=25,

```

```

seed=77)
    save_graph_to_json(graph, start=0, target=13, path=graph_path)
    return graph_path

def main() -> None:
    root = Path(__file__).resolve().parent
    results_dir = root / 'results'
    results_dir.mkdir(exist_ok=True)
    graph_path = ensure_demo_graph()
    graph, start, target = load_graph_from_json(graph_path)
    baseline = solve_with_dijkstra(graph, start, target)
    algorithm = ImmuneNetworkShortestPath(
        graph=graph,
        start=start,
        target=target,
        population_size=70,
        iterations=140,
        selection_size=18,
        clone_multiplier=5,
        mutation_rate=0.50,
        random_injection_rate=0.18,
        memory_size=10,
        seed=2026,
    )
    result = algorithm.run()
    deviation = percent_deviation(result.best_length, baseline.length)
    print('=== Искусственная иммунная сеть: кратчайший путь ===')
    print(f'Граф: {graph.vertices} вершин, {graph.edge_count} рёбер')
    print(f'Стартовая вершина: {start}')
    print(f'Конечная вершина: {target}')
    print(f'Путь ИС: {" -> ".join(map(str, result.best_path))}')
    print(f'Длина пути ИС: {result.best_length:.4f}')
    print(f'Эталон Дейкстры: {" -> ".join(map(str, baseline.path))}')
    print(f'Длина по Дейкстре: {baseline.length:.4f}')
    print(f'Отклонение: {deviation:.2f}%')
    print(f'Время выполнения ИС: {result.elapsed_time:.6f} сек')
    draw_graph_with_path(
        graph=graph,
        path=result.best_path,
        start=start,
        target=target,
        output_path=results_dir / 'path_visualization.png',
        title=f'ИС: путь {result.best_length:.1f}; Дейкстра: {baseline.length:.1f}',
    )
    plot_objective_history(
        history=result.history,
        output_path=results_dir / 'objective_vs_iterations_demo.png',
        optimal=baseline.length,
    )
    save_json(
        {
            'algorithm': 'Immune Network',
            'variant': '3 - shortest path in graph',
            'student': 'Сидоренко Максим',
            'graph_vertices': graph.vertices,
            'graph_edges': graph.edge_count,
            'start': start,
            'target': target,
            'immune_path': result.best_path,
            'immune_path_length': result.best_length,
            'dijkstra_path': baseline.path,
            'dijkstra_path_length': baseline.length,
            'deviation_percent': deviation,
            'elapsed_time_sec': result.elapsed_time,
            ...

```